

Yolk Survivors Guía técnica del rework

Arquitectura, sistemas y puntos de ajuste

8 de octubre de 2026 · Estado del código: 37060fa

Esta guía recorre los cambios de la jornada desde la creación de Fueguito. Explica cómo se conectan, qué scripts los implementan y qué valores conviene tocar para seguir desarrollando el juego. Está basada en el historial de Git y en la implementación actual, no en las propuestas que quedaron pendientes.

Commit	Cambios incluidos
8112920	Fueguito, ambientes, rediseño de armas, estadísticas y tienda.
b4a978f	Proyectiles únicos, tooltip de armas y corrección de Escape.
0a783aa	Diez personajes, estadísticas base, carga de pausa y alineación de UI.
09c38b5	Enemigos podridos y entidades del juego un 50% más grandes.
37060fa	Daño del player, velocidad de ataque, alcance, balance de armas y dificultad de enemigos.

Cómo usarla

Primero leé la arquitectura y el ciclo de partida. Después podés ir directamente al sistema que quieras cambiar. Cada capítulo separa la lógica en GDScript de los datos editables en el Inspector; al final hay tablas de balance, pruebas e índices de archivos.

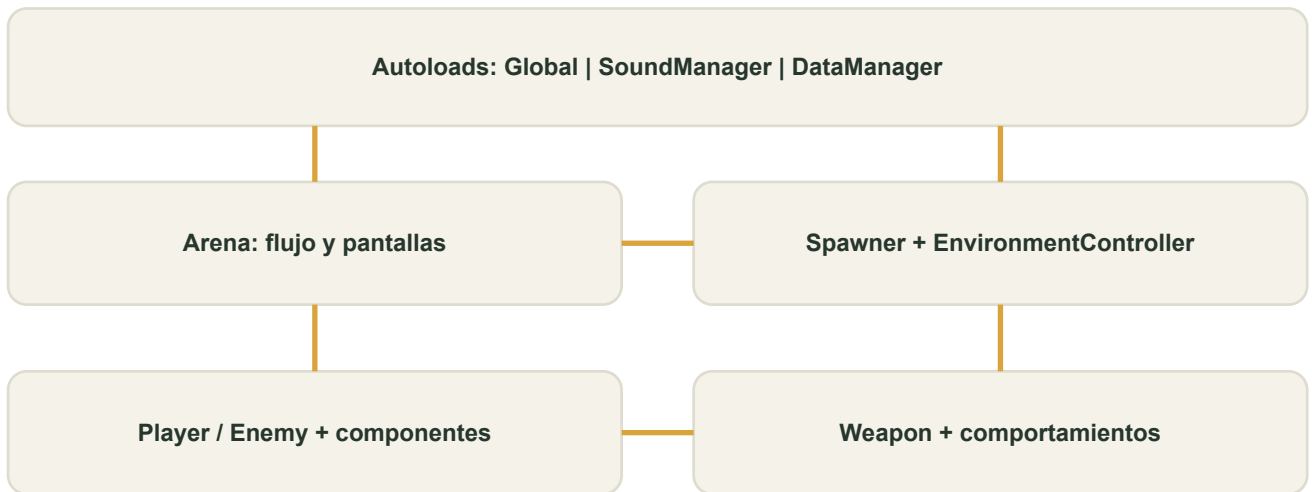
Alcance de la revisión. Comparación 062c7c6 → 37060fa. Las rutas res:// son relativas a la carpeta Godot yolk-survivors, ubicada dentro del repositorio. Se excluyen cachés de .godot y configuraciones locales del editor. Este trabajo agrega documentación; no cambia la lógica del juego.

01 / Mapa de lectura

Página	Tema
3	Arquitectura del código
4	Ciclo de partida y pausa
5	Fueguito: enemigo especial e invulnerabilidad
6	Ambientes, límites y cámara
7	Tienda, inventario y combinación
8	Tooltip y selección resaltada
9	Personajes y estadísticas de la partida
10	Cálculo de daño y mejoras ofensivas
11	Velocidad de ataque y prueba con Hidrante
12	Alcance y proyectiles propios
13	Reglas de dificultad
14	Spawner y escalado de enemigos
15	Rediseños, escala y ajustes de UI
16	Tabla actual de los diez personajes
17	Tabla actual de las armas
18	Recetas para retocar el juego
19	Scripts de verificación
20	Índice de scripts nuevos
21	Índice de scripts modificados: datos y combate
22	Índice de scripts modificados: UI y presentación

La numeración corresponde a esta edición. En los capítulos de sistemas se explica el comportamiento; los índices permiten encontrar rápidamente el archivo responsable.

02 / Arquitectura del código



El juego combina escenas (.tscn), scripts (.gd) y recursos (.tres). Una escena define nodos y conexiones; un script ejecuta el comportamiento; un recurso contiene la configuración reutilizable, como las estadísticas de un arma o la geometría de una arena.

Responsabilidades principales

Arena coordina selección, oleadas, mejoras, tienda, pausa y cierre de partida. Spawner decide cuándo y dónde aparecen enemigos, y prepara sus estadísticas. EnvironmentController aplica el ambiente correspondiente a la oleada.

Unit reúne la base compartida de las unidades. Player y Enemy especializan esa base. HealthComponent, HurtboxComponent y HitboxComponent separan vida, recepción de impactos y aplicación de daño. Fuegoito reutiliza Enemy y reemplaza las partes que permitirían dañarlo o destruirlo.

ItemWeapon es el recurso del arma; contiene WeaponStats, su escena, tipo y siguiente tier. Weapon es la instancia equipada; delega el ataque a RangeBehaviour o MeleBehaviour, que comparten WeaponBehaviour. El player mantiene sus instancias equipadas.

Global guarda selección, player activo, armas, monedas, dificultad y señales. SoundManager reproduce sonidos y configura buses. DataManager guarda volumen y dificultad desbloqueada en user://save.cfg. Estos dos últimos ya existían antes de hoy.

Regla práctica: para balancear, edita recursos .tres. Para cambiar qué ocurre, edita scripts .gd. Para reorganizar nodos, anclajes o conexiones, edita escenas .tscn. La UI vive en una capa separada del mundo y no usa la escala de sus entidades.

03 / Ciclo de partida y pausa

Selección → Dificultad → Oleada

Oleada terminada → Cosecha → Mejoras → Tienda → Oleada

Muerte / oleada final → Resultado → Nueva partida o salida

res://scenes/arena/arena.gd

Arena es el punto de entrada para entender la partida. La selección crea una copia de las estadísticas del personaje. La dificultad queda establecida antes de resetear e iniciar el Spawner, para que también afecte a la primera oleada.

Evento / señal	Reacción
Global.on_level_selected	Selecciona dificultad, resetea la partida e inicia oleada.
Spawner.on_wave_started	Aplica ambiente y límites correspondientes.
Spawner.on_wave_completed	Procesa cierre de oleada, cosecha y siguiente pantalla.
Global.on_upgrade_selected	Cierra mejoras y abre tienda.
ShopPanel.on_shop_next_wave	Prepara player e inicia siguiente oleada.
Global.on_enemy_died / on_player_died	Gestiona recompensas o resultado de derrota.

Corrección de Escape

Escape solo cambia la pausa cuando Arena está en la arena y wave_active está activo. En mejoras, tienda o transición no realiza acciones. Esto evita reanudar CookingPlayer por error después de una oleada.

La pausa se implementa con Global.game_paused y la pausa explícita de los timers del Spawner y de las pistas de audio; no es una pausa global del SceneTree. Los procesos de movimiento y combate consultan esa bandera. Fuegoito también detiene su animación.

Al extender el juego: un _process nuevo debe respetar Global.game_paused. Un await puede terminar durante una pausa; revisá el estado antes de ejecutar su continuación. La victoria actual se comprueba al completar la oleada 10 en arena.gd.

04 / Fueguito

Nuevo: `scenes/units/enemies/fueguito.gd`

Extiende Enemy. Conserva la persecución del player, pero reemplaza los callbacks de daño, knockback, muerte y destrucción por operaciones vacías. Es un peligro de contacto invulnerable, no una unidad que el jugador deba eliminar.

En `_ready` configura el Hitbox con su daño. En `_process` conserva el comportamiento heredado y ajusta `AnimationPlayer.speed_scale` a 0 durante la pausa y a 1 al reanudar.

Nuevo: `scenes/components/fueguito_health_component.gd`

Extiende HealthComponent y anula `take_damage` y `die`. Esta segunda protección cubre llamadas directas al componente, además de los callbacks anulados en el enemigo. Cambiar solo la vida numérica no desactivaría la invulnerabilidad.

Integración con Spawner y escena

`spawn_fueguito_if_needed` crea uno desde la oleada 2. Usa una distancia mínima de 450 unidades respecto al player, intentos aleatorios y una búsqueda alternativa por el perímetro. Si el espacio no permite una ubicación válida, puede omitir esa aparición.

Al terminar la oleada, `clear_fueguito` lo elimina explícitamente. En la siguiente oleada se crea otro. Su escena desactiva las capas del cuerpo y la hurtbox para que las armas no lo seleccionen como objetivo; el hitbox de contacto sigue funcionando.

Dónde retocar	Qué controla
<code>resources/units/enemies/stats_enemy_fueguito.tres</code>	Daño 5 y velocidad 150. Vida 1 nominal; no puede recibir daño.
<code>scenes/units/enemies/enemy_fueguito.tscn</code>	Sprite, animación, hitbox y componentes.
<code>scenes/arena/spawner.gd</code>	Oleada de entrada, distancia mínima y eliminación al cierre.

Fueguito queda fuera del escalado normal de dificultad, la selección aleatoria de enemigos y el límite común de unidades. Su arte aprobado se conserva. Los dos scripts nuevos son necesarios para mantener la inmunidad en ambos caminos de daño.

05 / Ambientes, límites y cámara

Nuevo: `resources/arena_environment.gd`

ArenaEnvironment es un Resource que agrupa `first_wave`, texturas de fondo y recipiente, tamaños, `offsets`, `movement_bounds`, forma oval, radio de esquinas y `camera_zoom`. Permite configurar una arena sin dispersar esos números entre nodos.

`clamp_position` devuelve una posición válida: limita un rectángulo, redondea sus esquinas o proyecta sobre una elipse. Limita el centro de la unidad. `random_spawn_position` genera puntos dentro de esa forma; en la elipse usa `sqrt(randf())` para distribuirlos por área.

Nuevo: `scenes/arena/environment_controller.gd`

Recibe un array de ArenaEnvironment. `apply_wave` elige el recurso con el `first_wave` más alto que no supere la oleada actual. Cambia texturas y adapta la escala visual al tamaño configurado. El orden del array no define la prioridad: la define `first_wave`.

Recurso actual	Entrada / geometría
<code>resources/environments/warm_skillet.tres</code>	Oleada 1. Rectángulo de 2000 × 900, esquinas 150, zoom 1.05.
<code>resources/environments/sage_casserole.tres</code>	Desde oleada 2. Elipse de 1480 × 1020, zoom 1.1.

Arena transmite el recurso elegido al player y al Spawner, ajusta el zoom y establece los límites de cámara a partir del fondo. `camera_2d.gd` sigue al player y limita su centro considerando el tamaño visible y el zoom; si la vista excede el fondo, la centra.

Agregar otro ambiente

Creá un ArenaEnvironment .tres, asigná texturas, dimensiones y `first_wave`, y agregalo al array del controlador en la escena Arena. Definí límites coherentes con el recipiente: no basta con reemplazar la imagen. Usá dimensiones positivas y un radio de esquinas adecuado al rectángulo.

El tamaño del fondo y el área jugable se configuran aquí. Son independientes del multiplicador 1.5 aplicado a personajes y elementos relacionados: agrandar una unidad no implica agrandar la arena.

06 / Tienda e inventario

```
res://scenes/ui/shop/shop_panel.gd
```

ShopPanel concentra la compra para que las tarjetas no cobren ni agreguen armas por caminos distintos. ShopCard entrega la acción a un `purchase_handler`; `try_purchase_item` valida player, coste, monedas y disponibilidad antes de modificar el inventario.

Compra con seis slots ocupados

Si el inventario está lleno, la compra solo puede continuar cuando existe un arma compatible que pueda subir de tier. `ItemWeapon.matches_for_upgrade` compara escena, tipo y tier, y exige `upgrade_to`. Se comprueba también que exista su tarjeta e instancia equipada.

La combinación automática reemplaza el arma existente por la versión siguiente, actualiza su tarjeta y el registro global, y cobra la compra. Si no hay combinación válida, no descuenta monedas. Con un slot libre, la compra agrega un arma normal; no combina automáticamente por tener una duplicada.

Tres representaciones que deben coincidir

Representación	Función
<code>Player.current_weapons</code>	Instancias reales que atacan.
<code>Global.equipped_weapons</code>	Recursos de las armas equipadas.
<code>WeaponContainer / ItemCard</code>	Slots visibles y acciones de la tienda.

La combinación manual consume dos armas y agrega su siguiente tier; actualmente su búsqueda usa `item_name`. La venta devuelve el 75% del precio, convertido a entero. Comprar un pasivo aplica sus estadísticas al player.

Para retocar: precios y cadenas `upgrade_to` están en `ItemWeapon.tres`. El límite de seis slots y la política de compra están en `ShopPanel`. Si cambiás el límite, revisá también las comprobaciones de capacidad y la presentación de los slots. Nunca agregues solamente una tarjeta: produciría un arma visible sin instancia de combate.

07 / Tooltip y selección resaltada

Nuevo: `scenes/ui/shop/weapon_tooltip.gd`

WeaponTooltip es un PanelContainer reutilizable. Construye una grilla de estadísticas y muestra daño, escalado, cooldown, alcance, crítico, knockback y robo de vida. Usa los cálculos efectivos de ItemWeapon con las estadísticas del player, por lo que refleja mejoras de la partida.

`show_for_card` valida que la tarjeta contenga un arma y muestra el panel. El proceso refresca sus valores y lo oculta si la tarjeta desaparece, se esconde o el mouse deja su zona. Su estilo deriva del tier y duplica el fondo antes de oscurecerlo.

`_position_above_card` intenta situarlo encima con 10 px de separación. Si no entra, usa la parte inferior; finalmente lo limita a los bordes con un margen de 12 px. Sus controles ignoran el mouse para no interrumpir el hover que lo activa.

ShopPanel crea una sola instancia y conecta `mouse_entered` / `mouse_exited` de los slots equipados. El tooltip no se agregó a todas las ofertas de la tienda. También se oculta al limpiar o cerrar el panel.

Nuevo: `scenes/ui/slot_selection_style.gd`

Es una utilidad RefCounted con métodos estáticos. `darkened` duplica un StyleBox y oscurece su fondo; `apply_button` configura estilos normal, hover y pressed. Al duplicar el estilo evita cambiar simultáneamente todos los controles que compartan el mismo recurso.

SelectionCard mantiene el resaltado del personaje y arma elegidos. ItemCard resalta el slot seleccionado, y las tarjetas de mejora y compra muestran su estado de selección. Los iconos no interceptan el mouse.

Dónde ajustar: contenido, separación y posicionamiento en `weapon_tooltip.gd`; apariencia común en `slot_selection_style.gd`; conexión de slots en `shop_panel.gd`; layout base en `weapon_tooltip.tscn`. El tooltip debe reutilizar las fórmulas de ItemWeapon para no mostrar un daño distinto al que se aplica.

08 / Personajes y datos de partida

```
res://resources/unit_stats.gd
```

El roster pasó a diez huevos: Well Rounded, Tiny Egg, Hardboiled, Mutant, Vampire, Gambler, Scrapper, Glass Egg, Hoarder y Berserker. Cada personaje tiene un UnitStats .tres, una escena de player y su sprite / retrato. Las identidades se expresan mediante números; no se agregó un sistema de habilidades exclusivas.

Global mantiene el mapa de escenas disponibles. SelectionPanel presenta el array de recursos y actualiza el personaje y arma elegidos. get_selected_player devuelve la escena correspondiente y una copia de las estadísticas, para que las mejoras de una partida no modifiquen el recurso base.

Separación del daño del player y del enemigo

UnitStats agrega damage_percent, melee_damage, ranged_damage y attack_speed. El campo damage y sus campos de crecimiento permanecen para el daño de los enemigos. El player ya no usa un daño plano universal para incrementar todas sus armas.

Vida, regeneración y cosecha

Se retiró el crecimiento de vida por oleada de los personajes. prepare_for_new_wave configura el HealthComponent con los valores actuales y recupera la vida. Esto no sube automáticamente la vida máxima. El timer de regeneración cura hp_regen cada 3 segundos, si corresponde y el juego no está pausado.

Harvesting entrega monedas al cerrar una oleada. No implementa crecimiento automático de cosecha ni experiencia. Las estadísticas de robo de vida siguen presentes, pero su lógica actual aún se ejecuta desde el ataque del arma.

Agregar un personaje

Creá el recurso en resources/units/players y su escena bajo scenes/units/players; asigná stats, sprite y retrato. Incorporalo al mapa de Global y al array de SelectionPanel en Arena. Actualizá la prueba del roster: hoy espera diez personajes.

Cambiar el recurso base afecta partidas nuevas. Para cambiar la partida que ya corre, el sistema aplica mejoras sobre Global.player.stats. La tabla de la página 16 resume los valores de esta edición.

09 / Daño, escalado y críticos

```
res://resources/items/weapons/item_weapon.gd
```

```
daño = max(1, (base + bono_tipo × escalado) × (1 + daño_% / 100))
bono_tipo = melee_damage o ranged_damage, según ItemWeapon.type
```

`get_effective_damage` concentra la fórmula. `WeaponStats.damage` es la base; `damage_scaling` define cuánto aporta el daño plano del tipo. Un escalado 0.50 significa que cada +1 de daño melee o ranged agrega 0.5 antes de aplicar Daño %. El mínimo de daño es 1.

Ejemplo: base 10, bono del tipo +8, escalado 50% y Daño +25%: $(10 + 8 \times 0.5) \times 1.25 = 17.5$. Un arma melee no usa `ranged_damage`. Se conservan decimales internamente; el formato de UI no redondea el cálculo del combate.

Críticos y comportamiento compartido

`WeaponBehaviour.get_damage` obtiene el daño efectivo y después aplica el multiplicador crítico del arma si la tirada acierta. Reinicia el estado de crítico en cada ataque para no arrastrarlo al siguiente. La probabilidad de crítico continúa siendo propia de `WeaponStats`, expresada como ratio de 0 a 1.

Mejoras y pasivos

Se incorporaron recursos de mejora para daño melee (+2 / +4 / +6 / +8), ranged (+1 / +2 / +3 / +4) y velocidad de ataque (+5 / +10 / +15 / +20). Las mejoras de Daño % usan el nuevo campo; los recursos existentes incluyen valores 5, 8, 12 y 16 según su rareza.

Los pasivos se migraron al daño porcentual: Ball aporta +10% y Rage +25%, con su penalización de velocidad. `ItemPassive` arma descripciones legibles con unidades correctas. Los nuevos recursos están referenciados en el pool de `upgrade_panel.tscn` para que puedan ofrecerse.

Archivos conectados: `unit_stats.gd` define los campos; `weapon_stats.gd` define base y coeficiente; `item_weapon.gd` calcula; `weapon_behaviour.gd` aplica crítico; los paneles llaman a la misma fórmula para mostrar resultados. La intención es que el balance tenga una sola fuente de cálculo.

10 / Velocidad de ataque

```
Si ataque_% >= 0: cooldown = base / (1 + ataque_% / 100)
Si ataque_% < 0: cooldown = base × (1 + abs(ataque_%) / 100)
cooldown final = max(0.05 segundos, cooldown)
```

La fórmula está en `ItemWeapon.get_effective_cooldown`. +100% reduce a la mitad el tiempo entre ataques, salvo que alcance el mínimo. -50% aumenta el cooldown base un 50%. `attack_speed` es un porcentaje, no una cantidad fija de disparos por segundo.

La animación no debe bloquear la cadencia

`Weapon.can_use_weapon` exige timer disponible, objetivo válido y que el arma no esté atacando. Por eso bajar solo el cooldown no alcanzaba: una animación larga podía seguir bloqueando el siguiente disparo.

`Weapon.get_animation_time_scale` ajusta las fases cuando su duración excede el 90% del cooldown efectivo. `RangeBehaviour` escala las dos fases de recoil; `MeleBehaviour` escala recoil, ataque y regreso. Así las armas rápidas terminan su movimiento antes del próximo ataque posible.

Hidrante como arma de alta cadencia

Dato	Valor actual
Cooldown base	0.08 s en los cuatro tiers
Daño I / II / III / IV	1 / 1 / 2 / 3
Escalado ranged	25%
Recoil	6 unidades; duración base 0.03 s por fase
Proyectil	Gota de mayonesa en color crema amarillento

Su cadencia nominal base es 12.5 ataques por segundo. La frecuencia real depende del frame, timer, objetivo y animación. El límite global de 0.05 s admite hasta 20 ataques por segundo nominales; no garantiza esa frecuencia en cualquier situación.

Para ajustar: cooldown, daño, `damage_scaling` y recoil están en los cuatro `WeaponStats` de Hidrante. Modificá todos los tiers cuando el cambio de identidad sea global; cambiá `attack_speed` del personaje si querés que el bono afecte todas sus armas.

11 / Alcance y proyectiles propios

res://scenes/projectiles/projectile.gd

Se unificaron detección y recorrido en `WeaponStats.max_range`. No se agregó otra estadística de distancia: el arma detecta enemigos dentro de su alcance y su proyectil desaparece al completar ese recorrido máximo.

`RangeBehaviour.create_projectile` pasa a `Projectile.setup` el alcance convertido a unidades del mundo mediante la escala de la colisión del arma. Con escala 1.5 y `max_range` 280, el recorrido es 420 unidades. La UI muestra el valor del recurso, no ese valor escalado.

Cómo se mide

`Projectile` guarda `max_distance` y `distance_travelled`. Cada frame suma la longitud de su desplazamiento real; si solo queda una fracción de recorrido, limita ese último paso y elimina el proyectil. La distancia parte de su punto de disparo, no de la posición actual del player: caminar no extiende su vida.

Durante la pausa no se mueve ni consume distancia. Todavía puede morir antes al impactar o salir de pantalla. El parámetro de recorrido es opcional y usa INF por defecto: los proyectiles enemigos conservan sus llamadas anteriores y dependen de impacto / salida de pantalla.

Identidad visual por arma ranged

Las cinco armas ranged usan escenas de proyectil distintas. `WeaponStats.projectile_scene` elige la escena y `projectile_speed` su velocidad. Las escenas `projectile_pistol`, `projectile_smg`, `projectile_laser`, `projectile_shotgun` y `projectile_revolver` comparten `projectile.gd`, pero cambian su presentación.

El proyectil de Hidrante se corrigió a crema amarillento para representar mayonesa. El rediseño de proyectiles no necesita duplicar la lógica de movimiento: la diferencia visual está en escena y textura.

Para retocar: `max_range` cambia detección y recorrido a la vez. `projectile_speed` cambia cuánto tarda en cubrirlo. `projectile_scene` cambia el aspecto. No hay todavía un atributo de alcance propio del player que se suma a todas las armas; el valor actual sigue siendo propio de cada arma.

12 / Reglas de dificultad

Nuevo: `resources/difficulty_rules.gd`

Es una utilidad `RefCounted`, cargada por los sistemas que la necesitan. Centraliza multiplicadores, oleadas de horda y textos explicativos; no es un autoload. `normalize_level` limita el nivel a 0-5; `stat_multiplier`, `is_horde` y `description` consultan las mismas reglas.

Nivel	Vida / daño	Hordas	Cambios adicionales
0	×1.00	Ninguna	Pool base
1	×1.00	Ninguna	Shooter y Charger desde oleada 3
2	×1.00	6	Entrada temprana de enemigos
3	×1.12	6	Entrada temprana de enemigos
4	×1.26	4, 6 y 9	Entrada temprana de enemigos
5	×1.40	4, 6 y 9	Entrada temprana de enemigos

Los multiplicadores son totales: el nivel 5 usa ×1.40, no apila los porcentajes de niveles anteriores. Afectan vida y daño; no aumentan automáticamente la velocidad del enemigo.

Panel de dificultad y guardado

`level_panel.gd` muestra las descripciones al pasar el mouse o enfocar un botón, y habilita los niveles según `Global.level_reached`. La selección entrega un ID 0-5. Arena lo normaliza antes de `reset_run` e iniciar la primera oleada.

`DataManager` conserva el mismo formato de guardado para la dificultad desbloqueada. Ganar desbloquea el siguiente nivel hasta el máximo 5. Los textos del panel se generan desde `DifficultyRules` para que no diverjan de los valores reales.

Oleadas de horda

En esas oleadas el intervalo normal se multiplica por 0.65: los intentos de aparición son más frecuentes. El pool especial favorece Fast (50%), Mid (37.5%) y Slow (12.5%). Sigue existiendo el límite de enemigos comunes vivos.

La dificultad se adaptó a partidas de diez oleadas. No hay todavía sistema de élités, bosses ni dificultad infinita. Para cambiar multiplicadores o calendario de hordas, edita las constantes de `difficulty_rules.gd` y revisa sus pruebas y los pools de oleadas.

13 / Spawner y escalado enemigo

```
res://scenes/arena/spawner.gd
```

```
vida = (vida inicial + crecimiento_vida × (oleada - 1)) × dificultad
daño = (daño inicial + crecimiento_daño × (oleada - 1)) × dificultad
```

prepare_enemy duplica las estadísticas y calcula el resultado antes de agregar la instancia al árbol. El HealthComponent recibe así la vida correcta desde _ready. Ya no se incrementan recursos compartidos al cerrar oleadas, evitando acumulaciones entre partidas y resultados que dependen del historial de spawns.

Enemigo	HP / +por oleada	Daño / +por oleada	Velocidad
Fast	6 / +2	2 / +0.35	350
Mid	10 / +4	3 / +0.45	250
Slow	20 / +9	4 / +0.60	150
Charger	12 / +5	5 / +0.75	230
Shooter	8 / +3	3 / +0.40	250

Ejemplo: Mid en oleada 4 y dificultad 3: HP = $(10 + 4 \times 3) \times 1.12 = 24.64$; daño = $(3 + 0.45 \times 3) \times 1.12 = 4.872$. Fuegoito no pasa por esta fórmula.

Pools y aparición segura

WaveUnitData agrega minimum_difficulty y first_wave. is_eligible descarta escenas nulas, pesos no positivos y entradas que todavía no correspondan. WaveData.get_random_unit_scene filtra primero y después sortea entre pesos relativos; los pesos no son porcentajes absolutos.

max_alive_enemies vale 80. pending_spawns reserva lugar mientras se muestra la advertencia; spawn_generation invalida apariciones que quedaron esperando al resetear o cambiar de oleada. El siguiente timer se programa antes del await de la advertencia, para que esa animación no determine toda la cadencia.

reset_run detiene y despausa timers, limpia unidades y vuelve a oleada 1. Después de un await se comprueba generación, estado de oleada, player y capacidad. No elimines estas validaciones al cambiar efectos de spawn: evitan enemigos tardíos o exceso de capacidad.

14 / Arte, escala y ajustes de UI

Rediseños aplicados

Se reemplazaron los personajes por el roster de diez huevos y sus recursos de selección. Los cinco enemigos comunes usan las versiones aprobadas con apariencia podrida: Charger toro, Fast veloz con orejas, Mid zombie, Slow ogro y Shooter con tres cañones grises. Fuegoito mantiene su diseño.

Las diez armas tienen sprites nuevos e iconos asociados; se retiraron los archivos antiguos. Los contornos y recursos visuales están vinculados desde sus escenas. Los cambios de arte no agregan por sí solos un comportamiento nuevo a cada enemigo o personaje.

Escala de mundo: +50%

```
res://autoload/global.gd
```

GAME_ENTITY_SCALE = 1.5 se aplica desde Unit a player y enemigos. Las armas y sus colisiones heredan la escala del player. Projectile, Coins y Fried la aplican en sus instancias independientes. El efecto de aparición tiene sus claves de animación ajustadas a esa escala.

PlayerTrail convierte la posición del player con to_local para no duplicar coordenadas mundiales bajo un padre escalado. La escala de las entidades también influye en hitboxes y alcance mundial; no es solamente un zoom cosmético del sprite.

El fondo, recipiente y UI no usan ese multiplicador. Ambiente y cámara mantienen sus propios valores. Para aumentar personajes, edita la constante; para cambiar área jugable, edita ArenaEnvironment.

Correcciones de presentación y carga

Se centraron verticalmente los valores +n de mejoras; Back y Continue quedaron alineados mediante sus anclajes; se regularizó el espaciado de estadísticas de player y arma. La selección y panel de stats se adaptaron a los nuevos campos ofensivos.

UpgradeCard calcula el tamaño del nombre para que etiquetas largas entren sin desbordarse. Se corrigieron las referencias de Arena a recursos de personajes eliminados: una escena que no carga puede provocar que PausePanel aparezca como null aunque el problema inicial sea un recurso ausente.

Si reaparece un error visible sobre null, verifica primero que arena.tscn cargue y que el nodo exista en su ruta. Añadir solo una comprobación de null no arregla una escena rota ni una referencia a un personaje eliminado.

15 / Estadísticas base de los huevos

Valores de recursos al iniciar una partida, antes de mejoras y pasivos. Daño % y ataque se expresan como porcentajes; melee y ranged son bonos planos. La velocidad es el valor del recurso, antes de considerar presentación y escala.

Huevo	HP	Daño %	Melee	Ranged	Ataque %
Well Rounded	20	10	2	2	5
Tiny Egg	14	-10	0	0	30
Hardboiled	36	0	5	0	-10
Mutant	22	5	3	3	0
Vampire	18	0	2	2	10
Gambler	16	0	0	2	5
Scrapper	24	0	4	1	10
Glass Egg	10	40	0	0	0
Hoarder	22	-10	0	0	0
Berserker	16	15	8	-2	15

Huevo	Vel.	Suerte	Block %	Regen / 3s	Robo %	Cosecha
Well Rounded	300	5	5	0	0	0
Tiny Egg	375	10	12	0	0	0
Hardboiled	250	0	22	0	0	0
Mutant	285	0	3	1	0	0
Vampire	310	0	0	0	12	0
Gambler	310	35	5	0	0	0
Scrapper	290	5	10	1	0	5
Glass Egg	325	0	0	0	0	0
Hoarder	275	15	5	0	0	15
Berserker	335	0	0	0	0	0

Lectura de identidad: Tiny favorece cadencia y movilidad; Hardboiled vida, bloqueo y melee; Vampire robo de vida; Gambler suerte; Glass Egg daño con poca vida; Hoarder cosecha; Berserker melee agresivo. Son especializaciones numéricas, no habilidades separadas.

Fuente: resources/units/players/player_*.tres. La ausencia de un campo en un .tres usa el valor por defecto de UnitStats; para estos bonos es 0. No se aplica crecimiento de vida por oleada a los personajes.

16 / Balance actual de armas

Valores base por tier I / II / III / IV. El daño final incorpora los bonos del player y el cooldown efectivo incorpora velocidad de ataque. El escalado se aplica solo al daño plano del tipo del arma.

Arma	Escalado	Daño I / II / III / IV	Cooldown I / II / III / IV
Blender	50%	5 / 8 / 12 / 18	0.50 / 0.48 / 0.45 / 0.42
Butcher Knife	80%	7 / 11 / 17 / 25	0.65 / 0.60 / 0.55 / 0.50
Mace	125%	12 / 19 / 29 / 43	1.20 / 1.10 / 1.00 / 0.90
Mitten	60%	6 / 10 / 15 / 22	0.55 / 0.50 / 0.45 / 0.40
Spatula	100%	8 / 13 / 20 / 30	0.90 / 0.85 / 0.80 / 0.75
Corpse Launcher	80%	9 / 14 / 21 / 31	0.80 / 0.80 / 0.80 / 0.80
Hidrante	25%	1 / 1 / 2 / 3	0.08 / 0.08 / 0.08 / 0.08
Knifer	60%	7 / 11 / 17 / 25	0.60 / 0.55 / 0.50 / 0.45
Smasher	125%	17 / 26 / 39 / 58	1.50 / 1.40 / 1.30 / 1.20
The Bloody	100%	12 / 19 / 29 / 43	1.00 / 0.95 / 0.90 / 0.85

Qué cambia cada parámetro

damage mueve el daño base del arma. damage_scaling cambia cuánto aprovecha las mejoras planas. cooldown cambia su cadencia. max_range cambia detección y recorrido ranged. crit_chance / crit_damage definen frecuencia y multiplicador de crítico. projectile_speed modifica el tiempo hasta el impacto.

Los tiers se configuran en recursos separados. Una mejora de daño base no requiere cambiar la fórmula; una nueva regla de daño sí. ItemWeapon.upgrade_to conecta los tiers que consume el sistema de combinación.

Fuente de esta tabla: docs/balance_armas.md y los WeaponStats actuales. El DPS base aproximado es daño / cooldown; no incluye crítico, fallos, tiempo de viaje, alcance, animación ni robo de vida. Un tier con mayor DPS de papel no garantiza mejor resultado en todas las situaciones.

17 / Recetas para seguir retocando

Quiero cambiar...	Dónde empezar	Qué revisar después
Daño de un personaje	resources/units/players/player_*.tres	Daño efectivo en selección y tooltip; melee / ranged correctos.
Identidad o cadencia de un arma	WeaponStats de sus cuatro tiers	Fórmula de daño, críticos, animación y cadena upgrade_to.
Distancia de proyectiles	WeaponStats.max_range	Mismo límite para detección y recorrido; escala 1.5.
Más enemigos por oleada	WaveData y Spawner	Intervalo, pesos, capacidad 80 y reservas pendientes.
Enemigos más resistentes	Stats enemigos y DifficultyRules	Crecimiento desde oleada 1; no modificar un recurso compartido.
Nuevo ambiente	ArenaEnvironment .tres + array del controlador	first_wave, límites, spawn, zoom y bordes de cámara.
Tooltip o resaltado	weapon_tooltip.gd / slot_selection_style.gd	Mouse, espacio disponible y estilos duplicados.
Cantidad de oleadas	arena.gd + recursos WaveData	Victoria en 10, calendario de hordas y pruebas.
Pausa en un sistema nuevo	Guardas de Global.game_paused	Timers y continuaciones de await; Escape solo en oleada.

Aspectos que aún no son una copia completa de Brotato

El rework implementado se centra en Daño %, melee, ranged, velocidad de ataque y escalado enemigo. No se agregaron ingeniería, daño elemental, armadura, esquiva ni una estadística global de alcance del player.

El robo de vida conserva su lógica anterior: se evalúa desde el ataque del arma y cura 1 según la probabilidad combinada, sin el cooldown compartido de 0.1 s. Cosecha entrega monedas sin crecimiento automático. Estas partes deben revisarse como sistemas propios si se quiere seguir acercándolas al referente.

18 / Scripts nuevos de verificación

Los cuatro scripts siguientes son pruebas headless, no forman parte del loop de juego. Sirven para detectar regresiones al retocar los sistemas. Esta revisión documental describe su cobertura; no ejecuta una nueva sesión de juego completa.

Script en tests/	Qué comprueba
verify_player_roster.gd	Roster de diez, copias de estadísticas, vida / regeneración / cosecha, transición de partida, Escape en oleada y fuera de ella, y presentación de stats.
verify_projectile_range.gd	Tiers ranged, igualdad entre detección y recorrido mundial, pausa, movimiento del player y eliminación en el límite; compatibilidad de proyectiles enemigos.
verify_offensive_stats.gd	Fórmula y decimales de daño, separación melee / ranged, mínimo 1, críticos, cooldown positivo / negativo, cap, mejoras y balance por tiers; cadencia y duración de animaciones.
verify_enemy_difficulty.gd	Escalado de cinco enemigos en diez oleadas y seis niveles, recursos base intactos, primera oleada, elegibilidad, hordas, capacidad concurrente, cancelación de spawns y excepción Fuegoito.

Ejecutar una prueba

```
Godot --headless --path "RUTA_AL_PROYECTO_GODOT"
--script res://tests/verify_offensive_stats.gd --quit-after 2000
```

El comando está dividido por legibilidad; en consola ejecútalo en una sola línea y reemplazá Godot por la ruta del ejecutable. La ruta del proyecto debe contener project.godot. Repetí el comando cambiando el nombre del script para el sistema que tocaste.

Validación manual que sigue siendo útil

Jugá una oleada, pausá y reanudá; abrí mejoras y tienda y probá Escape; comprá con seis armas; combiná y vendé; mirá el tooltip cerca de los bordes; compará un melee y un ranged tras mejorar daño. Probá distintas dificultades desde una partida nueva.

Las pruebas cubren reglas e integraciones concretas, no prueban por sí solas el balance de una partida completa ni reemplazan la revisión visual en distintas resoluciones. Separá los errores de lógica de avisos de cierre o recursos del editor al interpretar la salida.

19 / Índice: scripts nuevos de juego

Estos son los siete scripts runtime agregados durante la jornada. Los capítulos anteriores explican su integración; este índice resume dónde volver cuando quieras cambiar una responsabilidad.

Archivo	Responsabilidad / punto de ajuste	Página
resources/arena_environment.gd	Resource de arte, geometría y zoom. clamp_position y random_spawn_position.	6
resources/difficulty_rules.gd	Constantes y consultas compartidas de dificultad, hordas y descripción.	13
scenes/arena/environment_controller.gd	Elige el ambiente por first_wave y actualiza sprites del fondo y recipiente.	6
scenes/components/fueguito_health_component.gd	Bloquea daño y muerte mediante llamadas directas al componente.	5
scenes/ui/shop/weapon_tooltip.gd	Construye stats, actualiza valores y posiciona / oculta el tooltip de un slot.	8
scenes/ui/slot_selection_style.gd	Duplica y oscurece estilos para estados de botones y selección.	8
scenes/units/enemies/fueguito.gd	Enemy invulnerable: ignora daño, knockback y destrucción; pausa animación.	5

Archivos de configuración asociados

Las escenas enemy_fueguito.tscn y weapon_tooltip.tscn definen sus nodos. resources/environments contiene las dos arenas; resources/units/players contiene las diez bases; resources/units/enemies contiene el balance enemigo. Los recursos de armas e upgrades conectan esos datos con el combate y las ofertas.

No todo archivo nuevo es un sistema nuevo: sprites, escenas, .tres y archivos .uid acompañan la implementación. Los .uid identifican recursos en Godot; no contienen reglas de gameplay. Los cuatro scripts de tests están descritos en la página 19.

20 / Índice: datos y combate modificados

Archivo existente	Cambio de hoy
autoload/global.gd	Roster nuevo, estadísticas de selección y constante GAME_ENTITY_SCALE 1.5.
resources/unit_stats.gd	Nuevos campos ofensivos y separación de crecimiento enemigo.
resources/items/weapons/weapon_stats.gd	Coefficiente de daño y límites de cooldown / recoil.
resources/items/weapons/item_weapon.gd	Fórmulas efectivas, compatibilidad de combinación y descripción común.
resources/items/passives/item_passive.gd	Migración y presentación de bonos porcentuales.
resources/wave/wave_data.gd	Selección ponderada después de filtrar elegibilidad.
resources/wave/wave_unit_data.gd	Requisitos de dificultad y primera oleada por entrada.
scenes/arena/arena.gd	Flujo, pausa, nueva selección, ambientes y orden de dificultad.
scenes/arena/spawner.gd	Fueguito, escalado por instancia, hordas, capacidad y cancelación de spawns.
scenes/units/unit.gd	Escala compartida de entidades del mundo.
scenes/units/players/player.gd	Preparación de oleada y uso de estadísticas actuales sin crecimiento de HP.
scenes/weapons/weapon.gd	Cooldown efectivo y ajuste temporal de animación.
scenes/weapons/melee/weapon_behaviour.gd	Daño compartido, crítico y robo de vida.
scenes/weapons/melee/melee_behaviour.gd	Duración de fases según velocidad de ataque.
scenes/weapons/range/range_behaviour.gd	Recoil, escena de proyectil y recorrido ligado al alcance.
scenes/projectiles/projectile.gd	Escala, distancia recorrida y eliminación precisa al límite.

21 / Índice: UI y presentación modificadas

Archivo existente	Cambio de hoy
scenes/ui/level_panel/level_panel.gd	Descripción y desbloqueo de dificultad desde reglas comunes.
scenes/ui/selection_panel/selection_panel.gd	Roster y resaltado del personaje / arma seleccionados.
scenes/ui/selection_panel/selection_card.gd	Selección persistente y presentación de datos.
scenes/ui/shop/shop_panel.gd	Compra validada, combinación automática, inventario y tooltip.
scenes/ui/shop/shop_card.gd	Compra centralizada por callback y feedback de selección.
scenes/ui/shop/item_card.gd	Selección de slot y presentación de arma equipada.
scenes/ui/stats/stats_container.gd	Nuevos campos ofensivos y espaciado del panel.
scenes/ui/upgrades/upgrade_card.gd	Valor centrado, nombre ajustado y feedback de selección.
scenes/arena/camera_2d.gd	Límites según fondo, viewport y zoom del ambiente.
scenes/units/players/player_trail.gd	Coordenadas locales correctas después de escalar el player.
scenes/coins/coins.gd	Escala de moneda en el mundo.
coins_bag.gd	Parámetro <code>_delta</code> : limpieza de advertencia, sin cambiar comportamiento.
fried.gd	Escala del elemento visual relacionado con unidades.
shaders/outline.gdshader	Limpieza de espacios; sin cambio funcional del shader.

Escenas y recursos también importan

Las correcciones de anclajes, alineación y distribución viven en `.tscn`, aunque el script no muestre un cambio equivalente. Las sustituciones de personajes y enemigos incluyen textura de escena y referencias de recurso. Una búsqueda solo de `.gd` deja fuera parte de los cambios visuales.

Punto de partida recomendado: abrí `arena.tscn` para seguir el flujo, `unit_stats.gd` para entender los atributos y `item_weapon.gd` para entender el cálculo ofensivo. Después usé los capítulos específicos y ejecuté la prueba correspondiente al sistema que cambies.